

TRAITEMENT D'IMAGES

Compte Rendu No.3

Conception du Modèle pour la Reconnaissance des Émotions Faciales

Membres du Groupe

- Rayen Chemlali
- Mohamed Dhia Medini
- Khalil Ghimaji
- Mohamed Achref Hemissi

Informations

Filière	Génie Logiciel
Matière	Traitement d'Images
Semaine	3 — 13/04/2026
Dataset	FER2013

Contents

1 Introduction et Rappel du Problème 3

2 Choix de l'Approche : Deep Learning vs ML Classique 3

2.1 Comparaison des approches 3

3 Architectures du Modèle 4

3.1 Baseline CNN (architecture personnalisée) 4

3.1.1 Justification des choix de conception 5

3.2 ResNet-50 (Transfer Learning) 5

3.3 EfficientNet-B0 (Transfer Learning) 6

3.4 Comparaison des architectures 6

4 Correction des Statistiques de Normalisation 8

4.1 Problème identifié en Semaine 2 8

4.2 Correction apportée 8

5 Pipeline de Transformations et Augmentation 8

5.1 Deux pipelines distincts 9

5.2 Justification des augmentations 10

6 Gestion du Déséquilibre des Classes 12

6.1 Analyse du déséquilibre 12

6.2 Stratégie retenue : WeightedRandomSampler 12

6.2.1 Pourquoi ne pas combiner avec une loss pondérée ? 12

7 Configuration du DataLoader et Pipeline Global 13

7.1 Paramètres du DataLoader 13

7.2 Pipeline global 13

8 Configuration d'Entraînement 14

9 Fichiers Produits et Perspectives 14

9.1 Fichiers générés cette semaine 15

9.2 Perspectives — Semaine 4 15

1 Introduction et Rappel du Problème

L'objectif de ce projet est la **reconnaissance automatique des émotions faciales (FER)** : étant donné une image en niveaux de gris de résolution 48×48 pixels représentant un visage humain, prédire l'émotion parmi 7 classes : *Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral*.

Propriété	Valeur
Total images	35 887
Format	Niveaux de gris, 48×48 pixels
Ensemble train	28 709 images
Ensemble test	7 178 images
Classes	7 émotions

Table 1: Caractéristiques du dataset FER2013

Ce compte rendu présente les décisions de conception prises en semaine 3 : choix de l'approche deep learning, architecture des modèles, correction des statistiques de normalisation, pipeline d'augmentation et stratégie de gestion du déséquilibre de classes.

2 Choix de l'Approche : Deep Learning vs ML Classique

2.1. Comparaison des approches

Critère	ML Classique (SVM, RF + HOG/LBP)	Deep Learning (CNN)
Extraction de features	Manuelle (HOG, LBP, Gabor)	Apprise automatiquement
Précision sur FER2013	45–55 %	60–88 %
Variation d'éclairage	Mal gérée (filtres fixes)	Gérée (CLAHE + features apprises)
Déséquilibre des classes	Difficile à ajuster	Sampler + loss pondérée
Passage à l'échelle	Non	Oui

Table 2: Comparaison ML classique vs Deep Learning pour FER2013

Décision : Deep Learning

FER2013 en 48×48 est suffisamment petit pour entraîner un CNN léger *from scratch*, et suffisamment grand (28 K images d'entraînement) pour bénéficier des représentations apprises. Les meilleures performances publiées sur ce dataset sont obtenues avec des CNN profonds ($\sim 70\%$ de précision) contre $\sim 50\%$ pour les meilleures approches ML classiques. Trois architectures seront évaluées : **Baseline CNN**, **ResNet-50** et **EfficientNet-B0**.

3 Architectures du Modèle

3.1. Baseline CNN (architecture personnalisée)

Architecture légère conçue spécifiquement pour des images 48×48 en niveaux de gris, entraînée *from scratch*.

Couche	Opération	Sortie
Input	—	(B, 1, 48, 48)
Block 1	Conv(1→32, 3×3) + BN + ReLU + MaxPool(2)	(B, 32, 24, 24)
Block 2	Conv(32→64, 3×3) + BN + ReLU + MaxPool(2)	(B, 64, 12, 12)
Block 3	Conv(64→128, 3×3) + BN + ReLU + MaxPool(2)	(B, 128, 6, 6)
Block 4	Conv(128→256, 3×3) + BN + ReLU + MaxPool(2)	(B, 256, 3, 3)
GAP	GlobalAveragePool	(B, 256)
FC	Dropout(0.5) + Linear(256→7)	(B, 7)

Table 3: Flux spatial du Baseline CNN

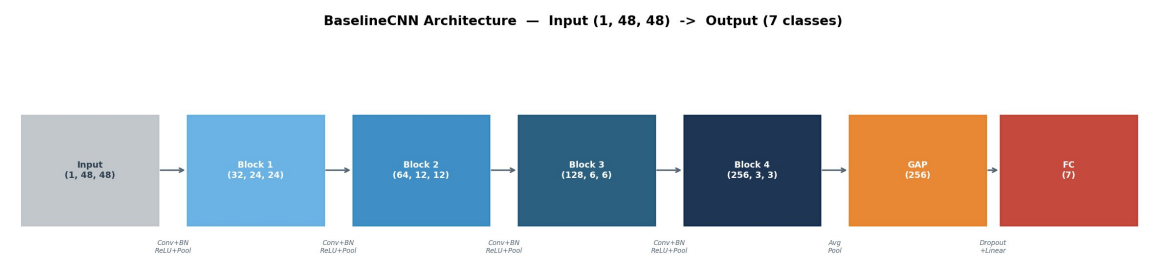


Figure 1: Diagramme de l'architecture BaselineCNN — entrée (1, 48, 48) → sortie (7 classes)

3.1.1. Justification des choix de conception

- **Conv(3×3) + padding=1** : conserve la résolution spatiale à chaque bloc avant le pooling. Un noyau 3×3 est le plus petit capable de capturer des relations spatiales entre pixels voisins.
- **BatchNorm après chaque Conv** : normalise les activations par batch, stabilise l'entraînement et permet des learning rates plus élevés. Indispensable car FER2013 contient des images issues de sources très variées (éclairage et contraste hétérogènes).
- **bias=False dans les Conv** : lorsque BatchNorm suit immédiatement, le biais de la convolution est annulé par le paramètre β appris par BN. Le supprimer évite une redondance et économise des paramètres.
- **MaxPool(2×2)** : divise la résolution par 2 à chaque bloc. Après 4 blocs : $48 \rightarrow 24 \rightarrow 12 \rightarrow 6 \rightarrow 3$. Agrandit progressivement le champ récepteur.
- **GlobalAveragePool (GAP)** : remplace Flatten + FC large. Moyenne chaque carte de features en un scalaire, produisant un vecteur de 256 valeurs. Aucun risque d'overfitting lié aux positions spatiales, et invariant à la taille d'entrée.
- **Dropout(0.5)** : régularisation avant la couche finale. Particulièrement utile vu la taille réduite du dataset.

Paramètres entraînaibles : 390 119 — architecture très légère, entraînable en quelques minutes sur CPU.

3.2. ResNet-50 (Transfer Learning)

ResNet-50 pré-entraîné sur ImageNet, adapté pour FER2013.

Adaptations nécessaires :

1. **Entrée 1 canal** : la première couche Conv est remplacée par une Conv(1→64). Les poids pré-entraînés RGB sont sommés sur la dimension des canaux pour initialiser les poids grayscale : $W_{\text{gray}} = W_R + W_G + W_B$. Cette initialisation préserve les features ImageNet.
2. **Suppression du MaxPool initial** : ResNet-50 standard applique stride=2 (Conv) puis MaxPool(2), ce qui réduirait 48×48 à 12×12 avant le premier bloc résiduel — trop agressif. Le MaxPool est remplacé par `nn.Identity()`.

Étape	Sortie
conv1 (stride=2)	(B, 64, 24, 24)
layer1 (pas de downsampling)	(B, 256, 24, 24)
layer2 (stride=2)	(B, 512, 12, 12)
layer3 (stride=2)	(B, 1024, 6, 6)
layer4 (stride=2)	(B, 2048, 3, 3)
GAP	(B, 2048)
Dropout(0.5) + FC	(B, 7)

Table 4: Flux spatial ResNet-50 adapté

3.3. EfficientNet-B0 (Transfer Learning)

EfficientNet-B0 pré-entraîné sur ImageNet, adapté de la même façon. La première Conv est remplacée par Conv(1→32, stride=2), avec initialisation des poids par somme des canaux RGB.

3.4. Comparaison des architectures

Modèle	Paramètres	Entraîné depuis	Avantage principal
Baseline CNN	390 119	Scratch	Léger, interprétable, rapide
EfficientNet-B0	4 015 939	ImageNet	Bon compromis taille/précision
ResNet-50	23 516 103	ImageNet	Features riches, précision maximale

Table 5: Comparaison des trois architectures

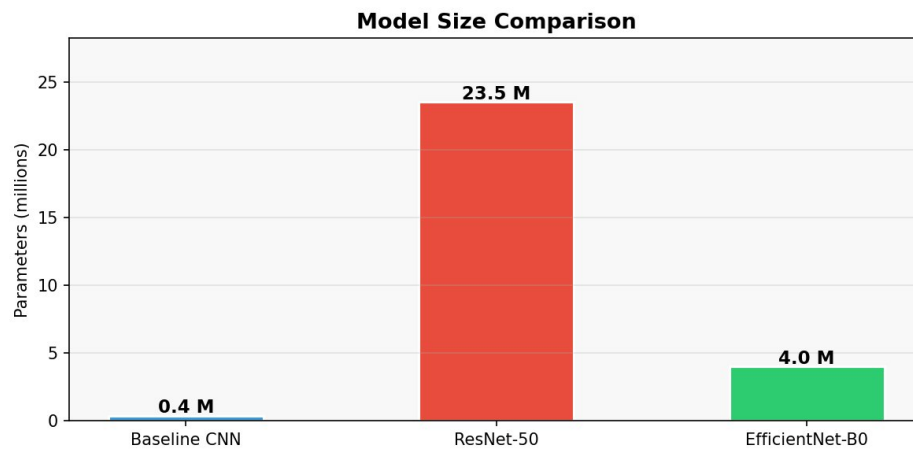


Figure 2: Comparaison du nombre de paramètres des trois architectures (en millions)

4 Correction des Statistiques de Normalisation

4.1. Problème identifié en Semaine 2

Les statistiques de normalisation calculées en CR2 ($\text{mean}=0.507$, $\text{std}=0.255$) étaient basées sur les **images brutes**, avant application de la CLAHE. Or dans le pipeline d'entraînement réel, la CLAHE est appliquée *avant* la normalisation, ce qui modifie significativement la distribution des valeurs de pixels.

Utiliser les anciennes statistiques produirait une distribution décalée en entraînement : le réseau ne recevrait pas des entrées centrées-réduites, dégradant la vitesse de convergence.

4.2. Correction apportée

Les statistiques sont recalculées sur le train set **après** l'chaîne complète (débruitage gaussien → CLAHE → normalisation $\div 255$) :

Source	Moyenne (μ)	Écart-type (σ)	Statut
Images brutes (CR2)	0.5077	0.2551	Incorrect
Après CLAHE (CR3)	0.5630	0.2630	Correct

Table 6: Correction des statistiques de normalisation

Règle fondamentale : les statistiques de normalisation doivent toujours être calculées *après* le pipeline de prétraitement complet, et *exclusivement* sur le train set pour éviter toute fuite de données (*data leakage*).

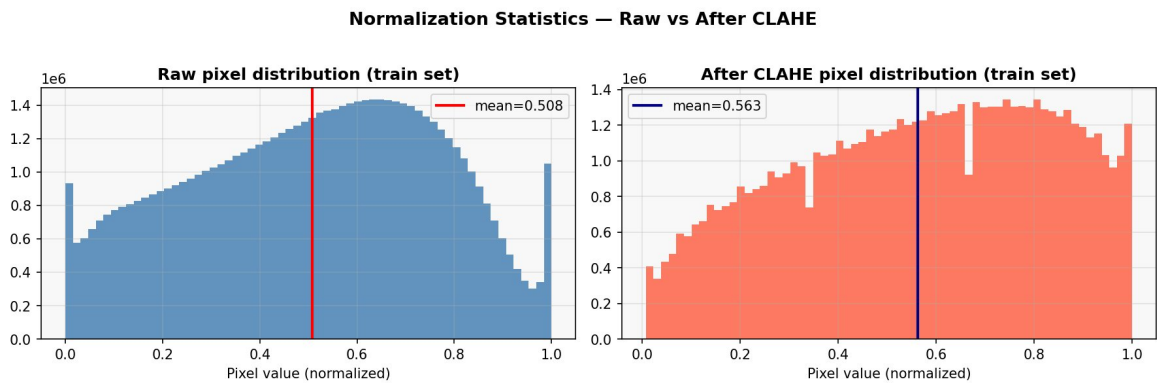


Figure 3: Distribution des pixels avant/après CLAHE — la moyenne passe de 0.508 à 0.563

5 Pipeline de Transformations et Augmentation

5.1. Deux pipelines distincts

	Entraînement (avec augmentation)	Validation / Test (sans augmentation)
1	GaussianDenoise (noyau 3×3, $\sigma=0.8$)	GaussianDenoise
2	CLAHE (clipLimit=2.0, tile=4×4)	CLAHE
3	RandomHorizontalFlip (p=0.5)	ToTensor
4	RandomRotation ($\pm 10^\circ$, fill=128)	Normalize(0.563, 0.263)
5	ColorJitter (brightness=0.3, contrast=0.2)	
6	RandomResizedCrop (48, scale=0.9–1.1, ratio=1.0)	
7	ToTensor	
8	Normalize(mean=0.563, std=0.263)	

Table 7: Pipelines de transformation entraînement vs validation/test

5.2. Justification des augmentations

Transformation	Justification	Paramètre
RandomHorizontalFlip	Les visages sont symétriques — le miroir ne change pas l'émotion	p=0.5
RandomRotation	Simule l'inclinaison naturelle de la tête	$\pm 10^\circ$
fill=128 (gris moyen)	Remplace les coins lors de la rotation — moins perturbateur que fill=0	128
ColorJitter brightness	Simule les variations d'éclairage	$\pm 30\%$
ColorJitter contrast	Simule les différences de qualité d'image	$\pm 20\%$
RandomResizedCrop	Légère variation de zoom ($\pm 10\%$ de surface)	scale=(0.9, 1.1)
ratio=(1.0, 1.0)	Force des crops carrés — évite la distorsion des proportions du visage	—

Table 8: Justification des paramètres d'augmentation

saturation et hue dans ColorJitter sont désactivés (None) car les images sont en niveaux de gris — ces transformations n'ont aucun effet et alourdiraient inutilement le pipeline.



Figure 4: Augmentation des données — image pré-traitée (sans augmentation) et 5 variantes augmentées par classe

6 Gestion du Déséquilibre des Classes

6.1. Analyse du déséquilibre

Classe	Effectif	Poids équilibré
Angry	4 953	1.027
Disgust	547	9.407
Fear	5 121	1.001
Happy	8 989	0.568
Sad	6 077	0.849
Surprise	4 002	1.293
Neutral	6 198	0.826

Table 9: Distribution des classes et poids équilibrés (rapport max Disgust/Happy = 1:16)

6.2. Stratégie retenue : WeightedRandomSampler

Le `WeightedRandomSampler` tire les images avec remplacement selon un poids proportionnel à l'inverse de la fréquence de leur classe. Disgust est ainsi tiré $\approx 16\times$ plus souvent que dans la distribution naturelle, produisant des batches quasi-équilibrés (≈ 9 images par classe dans un batch de 64).

6.2.1. Pourquoi ne pas combiner avec une loss pondérée ?

L'utilisation simultanée du sampler **ET** d'une `CrossEntropyLoss(weight=...)` produit une double correction :

Sampler seul : Disgust apparaît $16/7 \approx 2.3\times$ plus souvent par batch
 Loss pondérée : gradient de Disgust multiplié par $9.4\times$
 Combinaison : $2.3 \times 9.4 \approx 21\times$ -- sur-correction sévère

Cela conduirait le modèle à se spécialiser excessivement sur Disgust au détriment des classes majoritaires. La **loss non pondérée est donc correcte** lorsque le sampler est actif, car les batches sont déjà équilibrés.

Risque d'overfitting : Disgust (547 images) est répété $\approx 16\times$ par epoch. L'augmentation des données (flip, rotation, jitter, crop) est **essentielle** pour introduire de la variété dans ces images répétées.

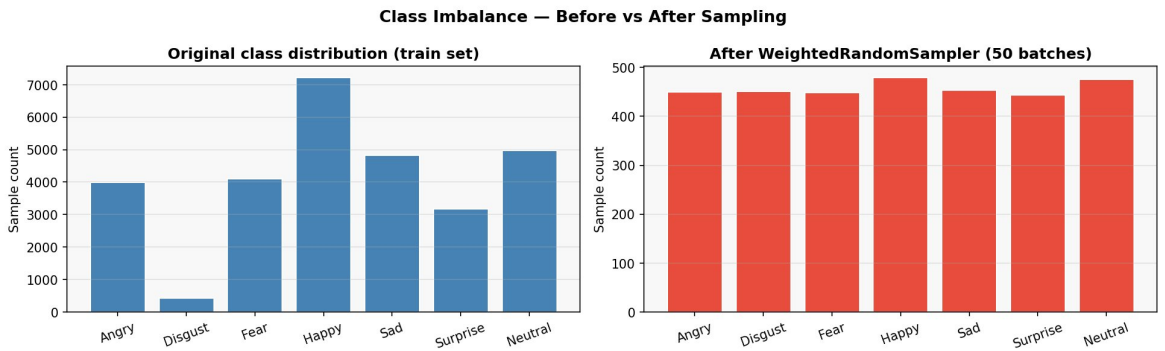


Figure 5: Distribution avant/après WeightedRandomSampler — les batches passent d’un déséquilibre 1:16 à une distribution quasi-uniforme

7 Configuration du DataLoader et Pipeline Global

7.1. Paramètres du DataLoader

Paramètre	Valeur
Batch size	64
Train loader	449 batches $\times$ 64 = 28 709 samples
Val/Test loader	113 batches $\times$ 64 = 7 178 samples
Sampler (train)	WeightedRandomSampler
Sampler (val/test)	Séquentiel (distribution naturelle)
Critère de perte	CrossEntropyLoss (non pondérée)

Table 10: Configuration du DataLoader

7.2. Pipeline global

Pipeline Complet d'Entraînement

- 1. **FERDataset** — détection automatique du format (CSV ou dossier), split train/val/test.
- 2. **Prétraitement** — GaussianDenoise → CLAHE (identique train et val/test).
- 3. **Augmentation** — RandomHorizontalFlip, RandomRotation, ColorJitter, RandomResizedCrop (train uniquement).
- 4. **Normalisation** — ToTensor → Normalize($\mu=0.563$, $\sigma=0.263$).
- 5. **WeightedRandomSampler** — équilibrage des classes dans chaque batch (train uniquement).
- 6. **Modèle** — (B, 1, 48, 48) → (B, 7), 3 architectures comparables.
- 7. **Optimisation** — Adam (lr=0.001, wd=0.0001), ReduceLROnPlateau, Early Stopping (patience=10).
- 8. **Évaluation** — accuracy globale, matrice de confusion, F1-score par classe.

8 Configuration d'Entraînement

Hyperparamètre	Valeur	Source
Optimiseur	Adam	Standard pour FER
Learning rate	0.001	config.yaml
Weight decay	0.0001	config.yaml
Epochs max	50	config.yaml
Early stopping patience	10	config.yaml
Batch size	64	config.yaml
Device	CUDA (auto)	config.yaml
Scheduler	ReduceLROnPlateau	Semaine 4

Table 11: Hyperparaètres d'entraînement

9 Fichiers Produits et Perspectives

9.1. Fichiers générés cette semaine

Fichier	Rôle
src/model.py	BaselineCNN, TransferModel, build_model()
src/transforms.py	GaussianDenoise, CLAHE, AddGaussianNoise
notebooks/cr3_model.ipynb	Notebook CR3 exécutable
configs/config.yaml	Mise à jour : mean=0.563, std=0.263
results/cr3_architecture.png	Diagramme BaselineCNN
results/cr3_model_comparison.	Comparaison du nombre de paramètres
results/cr3_normalization.png	Distribution avant/après CLAHE
results/cr3_augmentation.png	Exemples d'augmentation par classe
results/cr3_sampling.png	Distribution avant/après WeightedRandomSampler

Table 12: Fichiers produits en Semaine 3

9.2. Perspectives — Semaine 4

Prochaines Étapes

- **Entraînement complet** des trois architectures (Baseline CNN, ResNet-50, EfficientNet-B0) sur 50 epochs maximum avec early stopping.
- **Courbes d'apprentissage** — visualisation de la loss et de l'accuracy (train vs val) pour détecter l'overfitting.
- **Matrice de confusion** et **F1-score par classe** pour analyser les performances sur chaque émotion.
- **Sélection du meilleur modèle** basée sur la validation accuracy et la complexité du modèle.
- **Analyse des erreurs** — quelles émotions sont le plus souvent confondues (ex. Fear/Sad, Disgust/Angry) ?